

Complete Testing Guide - HyperPay Payment Gateway

Prerequisites

1. Database Setup

```
dotnet ef migrations add CompleteSetup
dotnet ef database update
```

2. Start the Application

```
dotnet run
```

3. Tools Required

- Postman or similar API testing tool
- Web browser for frontend testing

Phase 1: API Testing (Backend Only)

Test 1: Health Check

```
GET https://localhost:44383/health
```

Expected: Status 200, "Healthy"

Test 2: Register Application

```
POST https://localhost:44383/api/apiclient/register
Content-Type: application/json
```

```
{
  "clientName": "Test Ecommerce App"
}
```

Expected Response:

```
{
  "status": 1,
  "message": "Client registered successfully...",
  "data": {
    "clientId": 1,
    "clientName": "Test Ecommerce App",
    "apiKey": "abc123xyz..."
  }
}
```

Action: Copy the `apiKey` - you'll need it for all subsequent tests.

Test 3: Create Checkout Session

```
POST https://localhost:44383/api/payment/checkout
X-API-Key: YOUR_API_KEY_FROM_TEST_2
Content-Type: application/json
```

```
{
  "amount": 100.00,
  "merchantTransactionId": "ORDER-TEST-001",
  "paymentMethod": "CARD",
  "customer": {
    "email": "test@example.com",
    "givenName": "John",
    "surname": "Doe"
  },
  "billing": {
    "street1": "123 Main Street",
    "city": "Riyadh",
    "state": "Riyadh",
    "country": "SA",
    "postcode": "12345"
  }
}
```

Expected Response:

```
{
  "status": 1,
  "message": "Checkout session created successfully",
  "data": {
    "checkoutId": "xxx.yyy.zzz",
    "merchantTransactionId": "ORDER-TEST-001"
  }
}
```

Action: Copy the `checkoutId`.

What This Tests:

- API key authentication
- HyperPay API integration
- Database transaction creation
- Device/IP tracking

Test 4: Check Payment Status (Before Payment)

```
POST https://localhost:44383/api/payment/status
X-API-Key: YOUR_API_KEY
Content-Type: application/json
```

```
{
  "checkoutId": "YOUR_CHECKOUT_ID_FROM_TEST_3"
}
```

Expected: Status will be "PENDING" since no payment was made yet.

Phase 2: End-to-End Payment Testing (With Test Cards)

Step 1: Create Test HTML File

Save as `payment-test.html`:

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>HyperPay Payment Test</title>
<meta charset="UTF-8">
<style>
  body { font-family: Arial; max-width: 800px; margin: 50px auto; padding: 20px; }
  .section { border: 1px solid #ddd; padding: 20px; margin: 20px 0; border-radius: 5px; }
  .test-cards { background: #f0f8ff; padding: 15px; margin: 20px 0; }
  button { background: #007bff; color: white; padding: 10px 20px; border: none; border-radius: 5px; cursor: pointer; margin: 5px; }
  button:hover { background: #0056b3; }
  #result { background: #f8f9fa; padding: 15px; margin: 20px 0; white-space: pre-wrap; font-family: monospace; font-size: 12px; }
</style>
</head>
<body>
  <h1>HyperPay Payment Gateway Test</h1>

  <div class="section">
    <h2>Step 1: Create Checkout</h2>
    <label>API Key:</label><br>
    <input type="text" id="apiKey" style="width: 100%; padding: 8px; margin: 10px 0;" placeholder="Enter your API key">
    <br>
    <label>Amount (SAR):</label><br>
    <input type="number" id="amount" value="100.00" style="width: 200px; padding: 8px; margin: 10px 0;">
    <br>
    <button onclick="createCheckout()">Create Checkout Session</button>
  </div>

  <div class="test-cards">
    <h3>Test Card Details</h3>
    <p><strong>VISA (Success):</strong> 4012000033330026 | Expiry: 01/39 | CVV: 100</p>
    <p><strong>MasterCard (Success):</strong> 5123450000000008 | Expiry: 01/39 | CVV: 100</p>
    <p><strong>Card (Fail):</strong> 5204730000002514 | Expiry: 01/39 | CVV: 251</p>
    <p><strong>MADA (Success):</strong> 4464040000000007 | Expiry: 11/26 | CVV: 850</p>
    <p><em>Cardholder Name: Any Name</em></p>
  </div>

  <div class="section" id="paymentSection" style="display:none;">
    <h2>Step 2: Complete Payment</h2>
    <p>Checkout ID: <span id="checkoutIdDisplay"></span></p>
    <div id="paymentForm"></div>
  </div>

  <div class="section" id="statusSection" style="display:none;">
    <h2>Step 3: Verify Payment</h2>
    <button onclick="checkStatus()">Check Payment Status</button>
  </div>

  <div id="result"></div>

  <script>
    let checkoutId = '';
    let apiKey = '';

    async function createCheckout() {
      apiKey = document.getElementById('apiKey').value;
      const amount = parseFloat(document.getElementById('amount').value);

      if (!apiKey) {
        alert('Please enter API key');
        return;
      }

      const payload = {
        amount: amount,
        merchantTransactionId: 'ORDER-' + Date.now(),
        paymentMethod: 'CARD',
        customer: {
```

```

        email: 'test@example.com',
        givenName: 'John',
        surname: 'Doe'
    },
    billing: {
        street1: '123 Main St',
        city: 'Riyadh',
        state: 'Riyadh',
        country: 'SA',
        postcode: '12345'
    }
};

try {
    const response = await fetch('https://localhost:44383/api/payment/checkout', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-API-Key': apiKey
        },
        body: JSON.stringify(payload)
    });

    const result = await response.json();
    document.getElementById('result').textContent = JSON.stringify(result, null, 2);

    if (result.status === 1) {
        checkoutId = result.data.checkoutId;
        document.getElementById('checkoutIdDisplay').textContent = checkoutId;
        loadPaymentWidget();
        document.getElementById('paymentSection').style.display = 'block';
        document.getElementById('statusSection').style.display = 'block';
    }
} catch (error) {
    document.getElementById('result').textContent = 'Error: ' + error.message;
}

function loadPaymentWidget() {
    const script = document.createElement('script');
    script.src = `https://eu-test.oppwa.com/v1/paymentWidgets.js?checkoutId=${checkoutId}`;
    document.head.appendChild(script);

    script.onload = () => {
        document.getElementById('paymentForm').innerHTML = `
            <form action="payment-result.html?checkoutId=${checkoutId}&apiKey=${apiKey}" class="paymentWidgets" data-brands=
            `;
    };
};

async function checkStatus() {
    try {
        const response = await fetch('https://localhost:44383/api/payment/status', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'X-API-Key': apiKey
            },
            body: JSON.stringify({ checkoutId: checkoutId })
        });

        const result = await response.json();
        document.getElementById('result').textContent = JSON.stringify(result, null, 2);
    } catch (error) {
        document.getElementById('result').textContent = 'Error: ' + error.message;
    }
}

```

```

    }
  }
</script>
</body>
</html>

```

Step 2: Create Result Page

Save as `payment-result.html` :

```

<!DOCTYPE html>
<html>
<head>
  <title>Payment Result</title>
  <style>
    body { font-family: Arial; max-width: 600px; margin: 50px auto; padding: 20px; }
    .success { background: #d4edda; padding: 20px; border-radius: 5px; color: #155724; }
    .failed { background: #f8d7da; padding: 20px; border-radius: 5px; color: #721c24; }
    button { background: #007bff; color: white; padding: 10px 20px; border: none; border-radius: 5px; cursor: pointer; margin: 10px; }
    #details { background: #f8f9fa; padding: 15px; margin: 20px 0; white-space: pre-wrap; font-family: monospace; font-size: 12px; }
  </style>
</head>
<body>
  <h1>Payment Result</h1>
  <div id="status">Checking payment status...</div>
  <button onclick="checkAgain()">Refresh Status</button>
  <button onclick="window.location.href='payment-test.html'">New Payment</button>
  <div id="details"></div>

  <script>
    const urlParams = new URLSearchParams(window.location.search);
    const checkoutId = urlParams.get('checkoutId');
    const apiKey = urlParams.get('apiKey');

    async function checkStatus() {
      try {
        const response = await fetch('https://localhost:44383/api/payment/status', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            'X-API-Key': apiKey
          },
          body: JSON.stringify({ checkoutId: checkoutId })
        });

        const result = await response.json();
        document.getElementById('details').textContent = JSON.stringify(result, null, 2);

        if (result.status === 1) {
          const paymentStatus = result.data.paymentStatus;
          if (paymentStatus === 'SUCCESS') {
            document.getElementById('status').className = 'success';
            document.getElementById('status').innerHTML = `
              <h2>✓ Payment Successful!</h2>
              <p>Transaction ID: ${result.data.transactionId}</p>
              <p>Amount: ${result.data.amount} ${result.data.currency}</p>
            `;
          } else {
            document.getElementById('status').className = 'failed';
            document.getElementById('status').innerHTML = `
              <h2>✗ Payment Failed</h2>
              <p>${result.data.resultDescription}</p>
            `;
          }
        }
      } catch {
        // Handle error
      }
    }

    checkStatus();
  </script>
</body>
</html>

```

```
        } catch (error) {
            document.getElementById('details').textContent = 'Error: ' + error.message;
        }
    }

    function checkAgain() {
        checkStatus();
    }

    // Auto-check on load
    checkStatus();
</script>
</body>
</html>
```

Step 3: Run Complete Test

1. Open `payment-test.html` in browser
2. Enter your API key
3. Click "Create Checkout Session"
4. Enter test card details (use 4012000033330026 for success)
5. Complete payment
6. Verify status shows SUCCESS

Phase 3: Security Testing

Test 5: Rate Limiting

Make 65 requests within 1 minute to the same endpoint.

Expected: After 60 requests, you should get:

```
{
  "status": 0,
  "message": "Rate limit exceeded. Please try again later.",
  "data": null
}
```

Test 6: Invalid API Key

```
POST https://localhost:44383/api/payment/checkout
X-API-Key: invalid-key
```

Expected: 401 Unauthorized

Test 7: Missing Required Fields

```
POST https://localhost:44383/api/payment/checkout
X-API-Key: YOUR_API_KEY
Content-Type: application/json

{
  "amount": 100.00
}
```

Expected:

```
{
  "status": 0,
  "message": "MerchantTransactionId is required",
  "data": null
}
```

Phase 4: Admin Testing

Test 8: Global Statistics

```
GET https://localhost:44383/api/admin/stats/global
X-Admin-Key: admin_key_123@!#$
```

Test 9: View All Transactions

```
GET https://localhost:44383/api/admin/transactions/all?page=1&pageSize=10
X-Admin-Key: admin_key_123@!#$
```

Test 10: Stats by Application

```
GET https://localhost:44383/api/admin/stats/by-application
X-Admin-Key: admin_key_123@!#$
```

Test Card Scenarios

Card Number	Type	Expiry	CVV	Expected Result
4012000033330026	VISA	01/39	100	SUCCESS
5123450000000008	MasterCard	01/39	100	SUCCESS
5204730000002514	Card	01/39	251	FAILED
4464040000000007	MADA	11/26	850	SUCCESS

Important: Use whole amounts (100.00, 50.00) for successful test transactions.

Verification Checklist

After completing all tests, verify:

- ☐ Checkout sessions are created successfully
 - ☐ Transactions are saved in database with PENDING status
 - ☐ Payment widget loads correctly
 - ☐ Test cards process successfully
 - ☐ Payment status updates to SUCCESS/FAILED
 - ☐ Card details (last 4 digits, brand) are captured
 - ☐ Device information is recorded
 - ☐ Rate limiting works
 - ☐ API key authentication works
 - ☐ Admin endpoints require admin key
 - ☐ Each application sees only their transactions
 - ☐ Health check endpoint responds
-

Troubleshooting

Issue: "Invalid or inactive API Key"

- **Solution:** Ensure you're using the correct API key from registration

Issue: Payment status stays PENDING

- **Solution:** You must complete payment through the widget with test cards

Issue: "Rate limit exceeded"

- **Solution:** Wait 1 minute or restart the application

Issue: Database connection error

- **Solution:** Check PostgreSQL is running and connection string is correct
-

Production Readiness Checklist

Before deploying to production:

- ☐ Change `TestMode` to `false` in `appsettings.json`
 - ☐ Update to production HyperPay credentials
 - ☐ Change `AdminKey` to a secure random value
 - ☐ Store sensitive config in environment variables/Azure Key Vault
 - ☐ Update `AllowedOrigins` with production domains
 - ☐ Enable HTTPS only
 - ☐ Set up monitoring and logging
 - ☐ Configure database backups
 - ☐ Test with production cards (small amounts)
 - ☐ Set up webhook for real-time payment notifications
-

Next Steps

1. Integrate with your e-commerce frontend
2. Implement webhook handler for real-time updates
3. Add refund functionality
4. Set up monitoring and alerts
5. Configure automated backups

TMHyperPay — API Client Management Documentation

Audience: Backend developers, frontend integration team, QA
Base URL: `https://<your-host>/api/ApiClient`
Last Updated: 2026-03-19

Overview

Every system that wants to call the TMHyperPay payment APIs must first be registered as an **API Client**. When a client is registered, a unique **API Key** is generated. This key is passed with every payment request to prove the caller is authorized.

This section covers how to: - Register a new API client and get its key - List all registered clients - Regenerate a lost or compromised API key - Activate or deactivate a client

[!IMPORTANT] All endpoints in this controller require a valid **JWT Bearer Token** in the **Authorization** header. You must log in first via the Auth endpoints to get a token before calling any of these.

Authentication

Every request must include:

Authorization: Bearer <your_jwt_token>

If the token is missing or expired, the server returns HTTP 401 **Unauthorized** before your request even reaches the endpoint.

Standard Response Envelope

All responses — success or failure — follow the same wrapper shape:

```
{
  "success": true | false,
  "message": "Human-readable description",
  "data": { ... } // present only on success
}
```

Field	Type	Description
<code>success</code>	<code>bool</code>	<code>true</code> if the operation succeeded, <code>false</code> on any error
<code>message</code>	<code>string</code>	Describes the outcome or the reason for failure
<code>data</code>	<code>object</code>	The result payload. Only present when <code>success</code> is <code>true</code>

Endpoints

1. Register a New API Client

Why: Before any third-party system or service can use the HyperPay payment API, it must be registered. Registration creates a record in the database and generates a secure API key that the client will use to authenticate all future payment requests.

POST `/api/ApiClient/register`

Request Headers

Header	Value
Authorization	Bearer <jwt_token>
Content-Type	application/json

Request Body

```
{
  "clientName": "My Frontend App"
}
```

Field	Type	Required	Description
clientName	string	Yes	A human-readable name to identify this client (e.g. “Tamimi Mobile App”, “POS Terminal”). Max 100 characters.

Success Response 200 OK

```
{
  "success": true,
  "message": "Client registered successfully. Store this API key securely.",
  "data": {
    "clientId": 3,
    "clientName": "My Frontend App",
    "apiKey": "aB3xK9mNqR7vYz2..."
  }
}
```

Field	Description
clientId	The unique integer ID assigned to this client. Store this — you’ll need it to regenerate keys, toggle status, etc.
clientName	The name you submitted
apiKey	The raw, plain-text API key . This is the only time it will ever be shown. It is not stored in plain text in the database — only a secure hash is kept. Copy and store it immediately.

[!CAUTION] The `apiKey` in the registration response is shown **exactly once**. If you lose it, you cannot retrieve it — you must regenerate a new one. Treat it like a password.

Error Responses

Scenario	success	message
clientName is empty	false	"Client name is required"
Server error	false	"Error registering client: <details>"

2. List All API Clients

Why: Gives administrators a view of all registered clients — who they are, whether they are active, and when they last made a request. Useful for auditing and management UIs.

GET /api/ApiClient/list

Request Headers

Header	Value
Authorization	Bearer <jwt_token>

No request body needed.

Success Response 200 OK

```
{
  "success": true,
  "message": "Clients retrieved successfully",
  "data": [
    {
      "id": 1,
      "clientName": "My Frontend App",
      "apiKey": "****...aB3xK9mN",
      "isActive": true,
      "createdAt": "2026-03-15T08:30:00Z",
      "lastUsedAt": "2026-03-19T10:45:00Z"
    },
    {
      "id": 2,
      "clientName": "POS Terminal",
      "apiKey": "****...zQ8wP1rJ",
      "isActive": false,
      "createdAt": "2026-02-10T12:00:00Z",
      "lastUsedAt": null
    }
  ]
}
```

Field	Description
<code>id</code>	Unique ID of the client
<code>clientName</code>	Name given at registration
<code>apiKey</code>	Masked — shows *** followed by the last 8 characters of the stored hash. This is not the actual key; it's just for identification
<code>isActive</code>	true = client can make API calls. false = client is blocked
<code>createdAt</code>	UTC timestamp of when the client was registered
<code>lastUsedAt</code>	UTC timestamp of the last API call made by this client. null if never used

[!NOTE] The `apiKey` field in this response is intentionally masked for security. It is derived from the stored hash, not the original key. It cannot be used to authenticate. It is shown only to help identify which key is in use.

3. Regenerate API Key

Why: If an API key is lost, forgotten, or potentially compromised (e.g. accidentally committed to a repo), you can invalidate the old key and issue a brand new one without having to delete and re-create the entire client record. The old key **stops working immediately** the moment regeneration succeeds.

POST `/api/ApiClient/regenerate-api-key`

Request Headers

Header	Value
Authorization	<code>Bearer <jwt_token></code>
Content-Type	<code>application/json</code>

Request Body

```
{
  "clientId": 3
}
```

Field	Type	Required	Description
<code>clientId</code>	<code>int</code>	Yes	The ID of the client whose key you want to regenerate (obtained from the <code>register</code> or <code>list</code> responses)

Success Response 200 OK

```
{
  "success": true,
  "message": "API key regenerated successfully. Store this new key securely.",
  "data": {
    "clientId": 3,
    "clientName": "My Frontend App",
    "newApiKey": "xR7vYz2aB3mNqKp9...",
    "regenerationCount": 2,
    "regenerationsRemaining": 3,
    "lastRegeneratedAt": "2026-03-19T10:13:47Z"
  }
}
```

Field	Description
<code>clientId</code>	ID of the client
<code>clientName</code>	Name of the client
<code>newApiKey</code>	The new raw API key . Shown only once — save it immediately
<code>regenerationCount</code>	How many times this client's key has been regenerated so far (including this one)
<code>regenerationsRemaining</code>	How many more regenerations are still allowed before hitting the limit
<code>lastRegeneratedAt</code>	UTC timestamp of this regeneration

[!CAUTION] Just like registration, `newApiKey` is shown **only once**. If you lose it, you'll need to regenerate again (consuming another slot from your limit).

Regeneration Limit Each client has a **maximum regeneration limit** (default: **5**). This is a safety guard to prevent abuse — if someone keeps regenerating keys, it creates a natural audit trail and eventually gets blocked.

- When the limit is **reached**, the endpoint returns a failure response and the old key remains valid.
- To increase the limit, an administrator must update it directly in the database (or via an admin endpoint).

Error when limit is reached:

```
{
  "success": false,
  "message": "Maximum regeneration limit of 5 has been reached for this client. Contact an admin."
}
```

All Error Responses

Scenario	success	message
clientId not found	false	"Client with ID X not found"
Regeneration limit reached	false	"Maximum regeneration limit of N has been reached..."
Server error	false	"Error regenerating API key: <details>"

4. Toggle Client Status (Activate / Deactivate)

Why: Sometimes you need to temporarily block a client from making API calls — for example, if you suspect misuse, or if the client's subscription has expired — without permanently deleting their record. This toggle flips between active and inactive.

POST /api/ApiClient/toggle

Request Headers

Header	Value
Authorization	Bearer <jwt_token>
Content-Type	application/json

Request Body

```
{
  "clientId": 3
}
```

Field	Type	Required	Description
clientId	int	Yes	ID of the client to activate or deactivate

How it works This endpoint reads the **current isActive** value and flips it:
- If currently **true** → sets to **false** (deactivates) - If currently **false** → sets to **true** (activates)

You do **not** pass the desired state — it always toggles.

Success Response 200 OK

```
{
  "success": true,
  "message": "Client deactivated successfully",
  "data": {
    "clientId": 3,
    "isActive": false
  }
}
```

Field	Description
clientId	The ID of the affected client
isActive	The new status after the toggle

Error Responses

Scenario	success	message
clientId not found	false	"Client not found"
Server error	false	"Error toggling client status: <details>"

[!NOTE] When a client is deactivated (**isActive: false**), any API calls made with their key will be rejected by the authentication middleware — they do not need to be deleted, just toggled off.

Database Model Reference

The `ApiClient` table stores the following fields:

Column	Type	Default	Description
Id	int	Auto-increment	Primary key
ClientName	string	—	Display name of the client
ApiKeyHash	string	—	Bcrypt/HMAC hash of the API key. The actual key is never stored
ApiKeySalt	string	—	Salt used during hashing
IsActive	bool	true	Whether this client is allowed to make API calls
CreatedAt	DateTime	NOW()	When the client was registered
LastUsedAt	DateTime?	null	When the client last made a successful API call
UserId	int?	null	ID of the admin user who registered this client
MaxRegenerations	int	5	Maximum number of times the API key can be regenerated
RegenerationCount	int	0	Running counter of how many regenerations have occurred
LastRegeneratedAt	DateTime?	null	UTC timestamp of the most recent key regeneration
LastRegeneratedByIp	string?	null	IP address that triggered the most recent regeneration

Audit Log Events

Every important action is written to the audit log automatically. Here are the event codes you will see:

Event Code	Trigger
CLIENT_REGISTERED	A new client was successfully registered
CLIENT_REGISTRATION_FAILED	Registration was attempted but failed (e.g. missing name)
CLIENT_REGISTRATION_ERROR	Unexpected server error during registration
API_KEY_REGENERATED	API key was successfully regenerated
API_KEY_REGEN_FAILED	Regeneration attempted for a non-existent client
API_KEY_REGEN_LIMIT_REACHED	Regeneration blocked because MaxRegenerations was reached
API_KEY_REGEN_ERROR	Unexpected server error during regeneration
CLIENT_STATUS_CHANGED	Client was activated or deactivated
CLIENT_TOGGLE_FAILED	Toggle attempted for a non-existent client
CLIENT_TOGGLE_ERROR	Unexpected server error during toggle

Frontend Integration Guide

Typical Workflow

```
sequenceDiagram
    participant Admin as Admin UI
    participant API as TMHyperPay API
    participant DB as Database

    Admin->>API: POST /api/Auth/login
    API-->>Admin: JWT Token

    Admin->>API: POST /api/ApiClient/register { clientName }
    API->>DB: Create ApiClient row, store hash
    API-->>Admin: { clientId, apiKey } ← COPY THIS KEY NOW

    Admin->>API: GET /api/ApiClient/list
    API-->>Admin: All clients (masked keys)

    Admin->>API: POST /api/ApiClient/toggle { clientId }
```

```
API-->>Admin: { isActive: false }
```

```
Admin-->>API: POST /api/ApiClient/regenerate-api-key { clientId }
```

```
API-->>Admin: { newApiKey } ← COPY THIS KEY NOW
```

Key Points for the Frontend Team

1. **Always store the API key immediately** after register or regenerate-api-key. Display it in a modal with a copy button and a warning. There is no “show again” button.
2. **clientId is your primary handle** for all subsequent operations. Store it alongside the client name in your frontend state after registration.
3. **The apiKey field in list is masked** — it’s not usable for authentication. Don’t confuse it with the real key.
4. **Toggle is stateless** — you don’t tell it “activate” or “deactivate”. It just flips. So always read `isActive` from the response to know the new state, and update your UI accordingly.
5. **Show regenerationsRemaining** in your admin UI so admins know how many regenerations are left before hitting the cap. Warn when it’s at 1.
6. **All requests require a JWT token** — implement a request interceptor in your HTTP client to attach the `Authorization: Bearer <token>` header on every call.

Example: Register + Show Key (JavaScript/Fetch)

```
async function registerClient(jwtToken, clientName) {  
  const response = await fetch('/api/ApiClient/register', {  
    method: 'POST',  
    headers: {  
      'Authorization': `Bearer ${jwtToken}`,  
      'Content-Type': 'application/json'  
    },  
    body: JSON.stringify({ clientName })  
  });  
  
  const result = await response.json();  
  
  if (result.success) {  
    const { clientId, clientName, apiKey } = result.data;  
    // Show apiKey to the user NOW - it won't be available again  
    showApiKeyModal(clientId, clientName, apiKey);  
  } else {  
    showError(result.message);  
  }  
}
```

```
}  
}
```

Example: Regenerate Key (JavaScript/Fetch)

```
async function regenerateKey(jwtToken, clientId) {  
  const response = await fetch('/api/ApiClient/regenerate-api-key', {  
    method: 'POST',  
    headers: {  
      'Authorization': `Bearer ${jwtToken}`,  
      'Content-Type': 'application/json'  
    },  
    body: JSON.stringify({ clientId })  
  });  
  
  const result = await response.json();  
  
  if (result.success) {  
    const { newApiKey, regenerationsRemaining } = result.data;  
    // Show newApiKey to the user NOW  
    showApiKeyModal(clientId, null, newApiKey);  
    updateRegenerationCounter(clientId, regenerationsRemaining);  
  } else {  
    showError(result.message); // Might be limit reached  
  }  
}
```